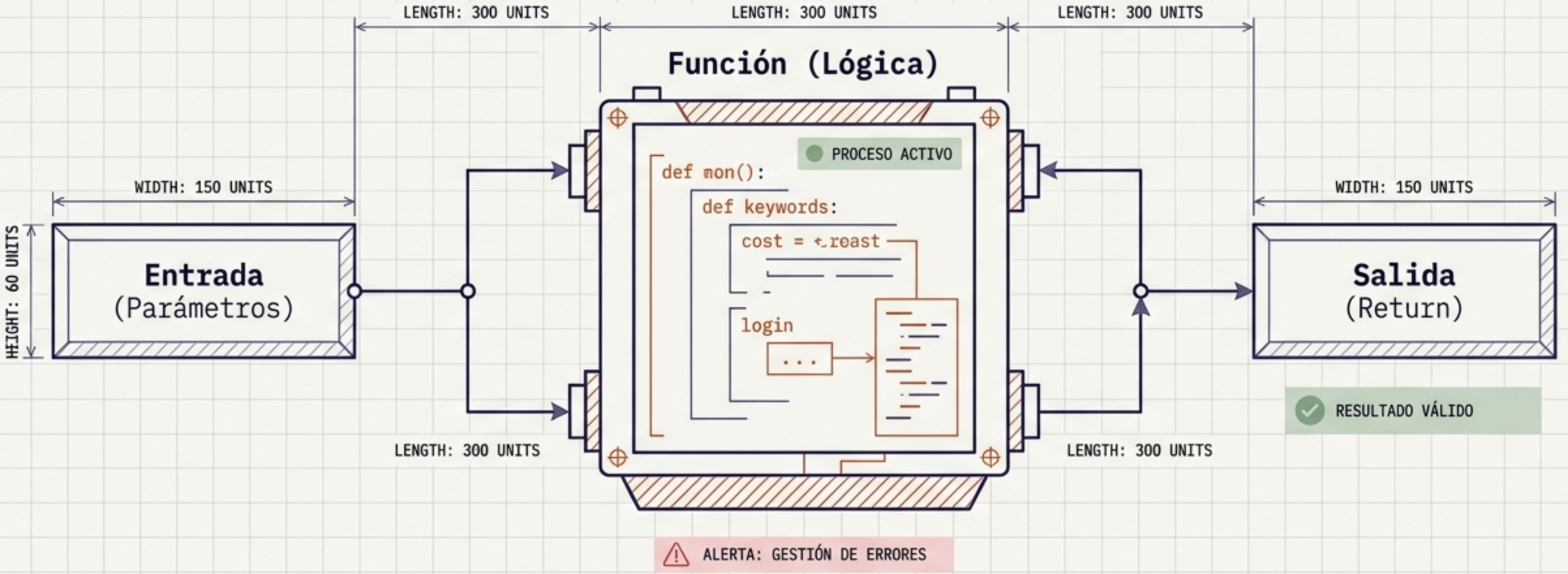
Funciones en Python: El Plano Arquitectónico

Guía visual definitiva de sintaxis, parámetros y arquitectura de código limpio.



SCALE 1:1 | DRAWN BY: SYSTEM ARCHITECT

Gemini Notebook

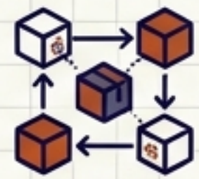
¿Qué es una función y por qué importa?

Las funciones son bloques de código reutilizables que encapsulan una tarea específica. Piensa en ellas como máquinas de un solo propósito dentro de tu programa.



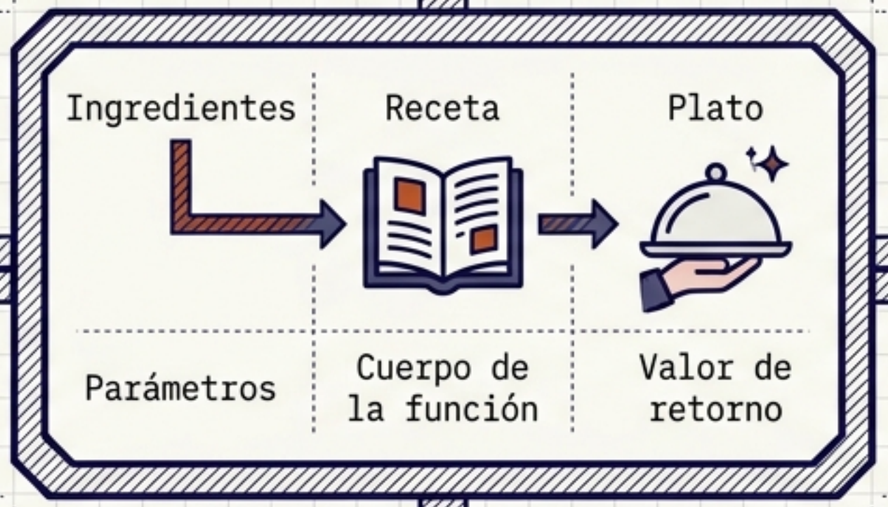
Reutilización

Escribe una vez, usa muchas veces. Elimina la repetición de código.



Organización

Divide problemas monolíticos y complejos en **partes más pequeñas** y manejables.





Mantenibilidad

Los **cambios o correcciones** en un solo lugar **actualizan automáticamente** todos los usos de esa función.

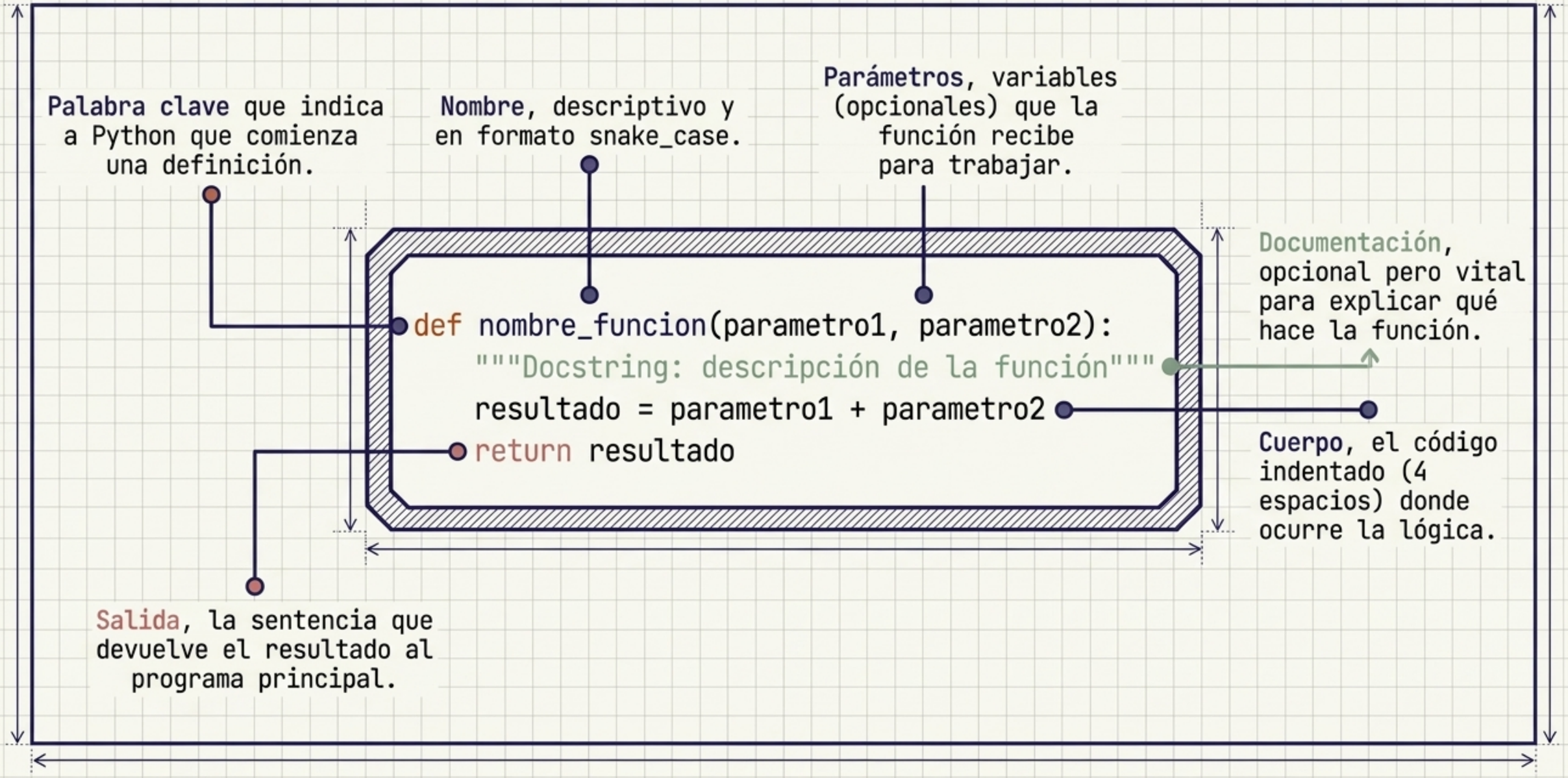


Testing y Legibilidad

Código más claro, **autodocumentado** y **fácil de probar** en unidades aisladas.

Gemini Notebook

Anatomía de una función en Python



Gemini Notebook

El Sistema de Entradas: Tipos de Parámetros

Posicionales

El orden dicta la regla.

```
calcular_area(5, 10)
```

Los valores se asignan en el orden exacto en que se definieron.

Con Nombre (Keyword)

La claridad es el rey.

```
crear_usuario(nombre="Ana",
              edad=25)
```

Permite alterar el orden. Mejora drásticamente la legibilidad en llamadas complejas.

Por Defecto

Valores seguros de respaldo.

```
def saludar(nombre,
    saludo="Hola"):
```

Si el usuario no envía ese dato, la función usa el valor predeterminado.



REGLA ESTRICTA: Los parámetros con valores por defecto **siempre deben ir después de los parámetros obligatorios** (SyntaxError garantizado si no se cumple).

Gemini Notebook

El Sistema de Salidas: La Sentencia `return`



Return simple

Devuelve un solo valor.

```
return a + b
```

Return múltiple

Devuelve una tupla que se puede desempaquetar.

```
return total, promedio, maximo
```

```
suma, prom, max_val = calcular_estadisticas(nums)
```

Sin return

Finaliza y devuelve None implícitamente (ej. funciones que solo hacen print()).

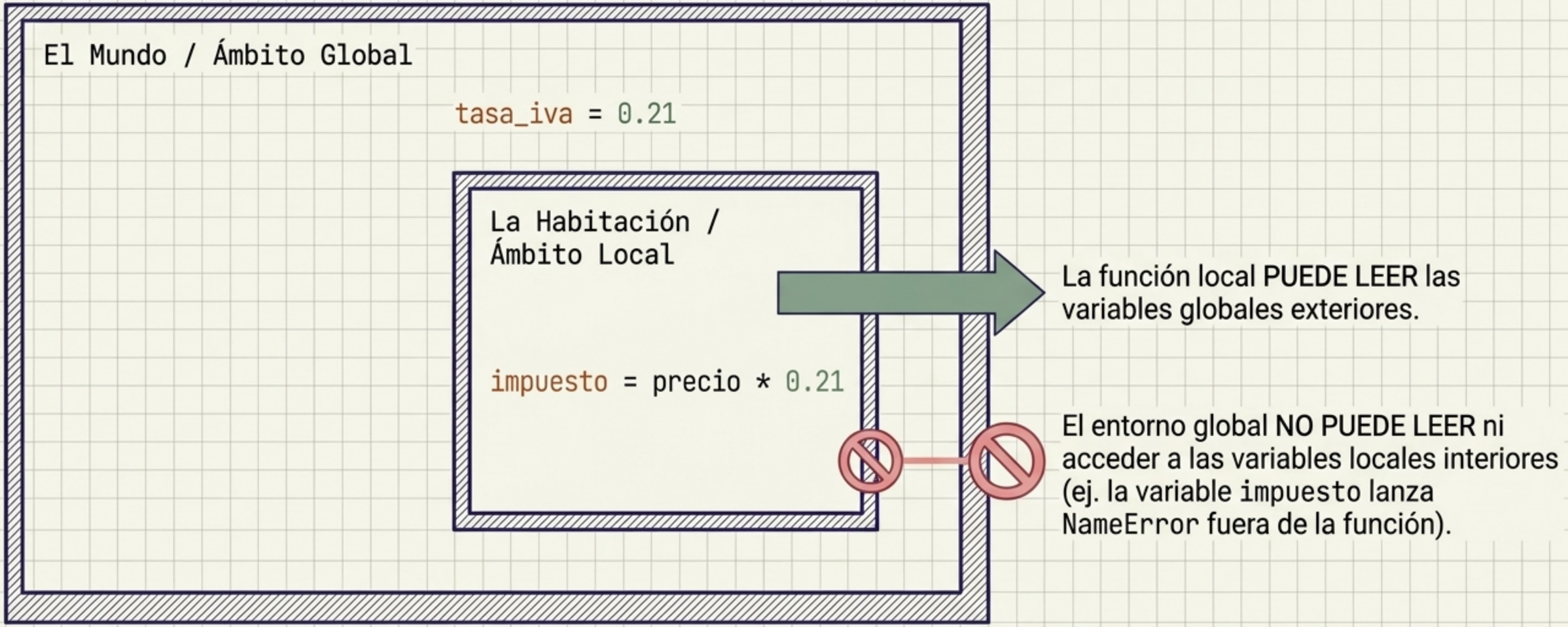
Return anticipado

Cortocircuito lógico. Sale de la función antes de tiempo si se cumple una condición.

```
if num % 2 == 0:
    return True
```

Gemini Notebook

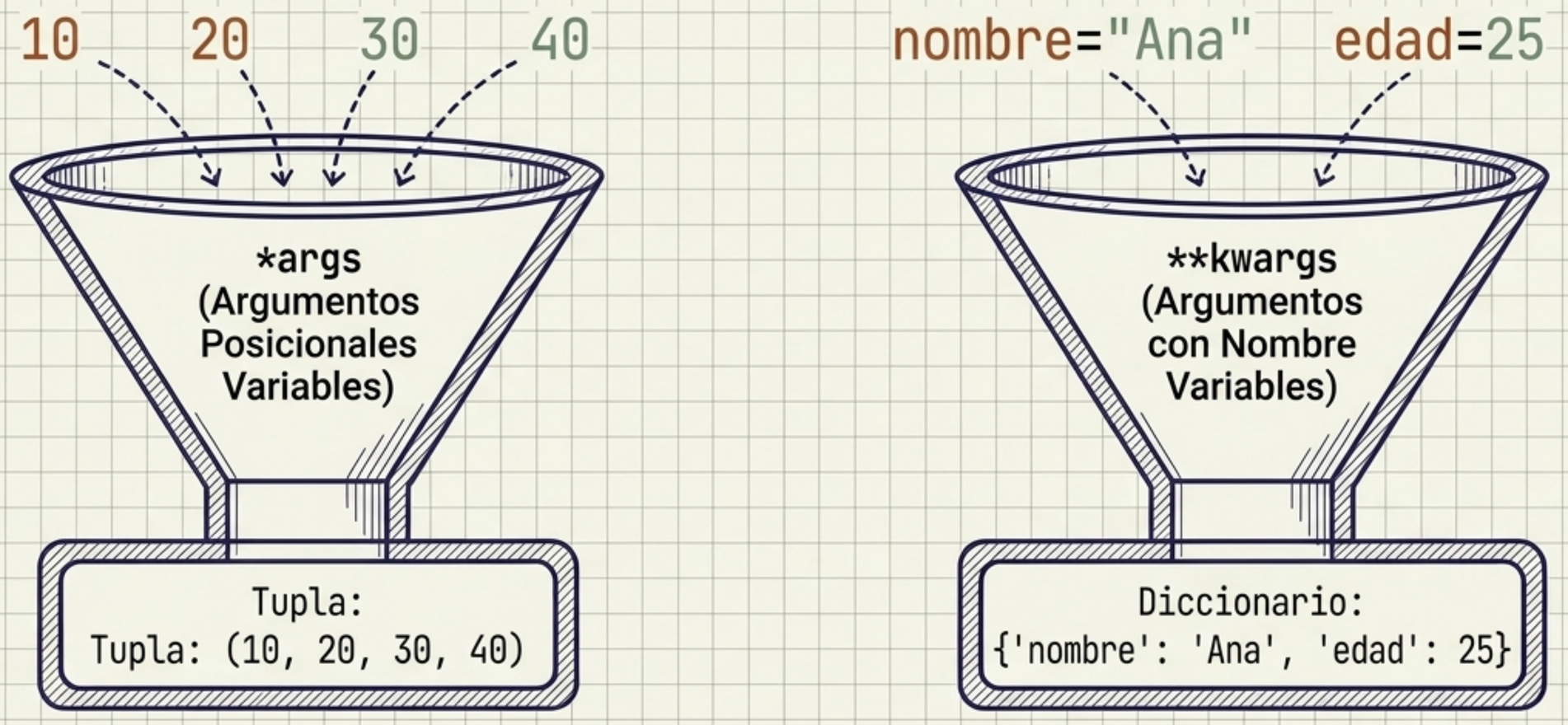
Scope: El Mapa de Acceso a Variables



Para modificar una variable global desde dentro de la función local, se requiere declarar global nombre_variable, pero es una mala práctica. Lo ideal es pasar datos por parámetros y sacarlos con return.

Gemini Notebook

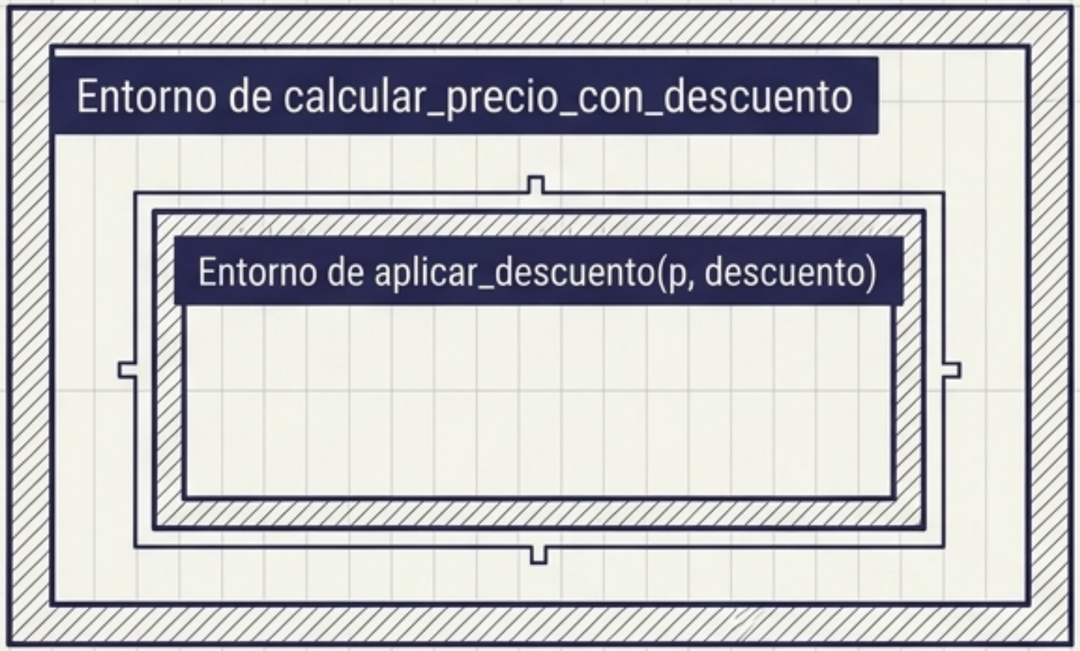
Flexibilidad Extrema: *args y **kwargs




```
def funcion_completa(obligatorio, opcional="default", *args, **kwargs):
```

Gemini Notebook

Funciones Anidadas y Closures (Inception de Código)



- **Concepto Clave**: Puedes definir funciones dentro de otras funciones. Se usan principalmente para encapsular lógica auxiliar que solo tiene sentido dentro de la función principal.
- **Closures**: Las funciones internas tienen memoria y acceso al entorno de la función externa.

```
def crear_multiplicador(factor):
    def multiplicar(numero):
        return numero * factor  # Accede a 'factor'
    return multiplicar

duplicar = crear_multiplicador(2)
```

Gemini Notebook

El Estándar de Oro: Docstrings

Resumen:
Descripción clara y directa.

Argumentos (Args):
Define tipo de dato y propósito de cada entrada.

```
def calcular_imc(peso, altura):
    """
    Calcula el Índice de Masa Corporal (IMC).

    Args:
        peso (float): Peso en kilogramos
        altura (float): Altura en metros

    Returns:
        float: El IMC calculado

    Examples:
        >>> calcular_imc(70, 1.75)
        22.86
    """
```

Retornos (Returns):
Define qué tipo de dato devuelve.

Ejemplos: Código ejecutable demostrativo.

Gemini Notebook

El Secreto de Python: Funciones como Objetos de Primera Clase

Objeto Función

Asignar
Pueden asignarse a variables.

```
mi_saludo = saludar
```

Pasar
Pueden pasarse como argumentos a otra función.

```
aplicar_operacion(5, 3, sumar)
```

Retornar
Pueden retornarse como resultado de otra función.

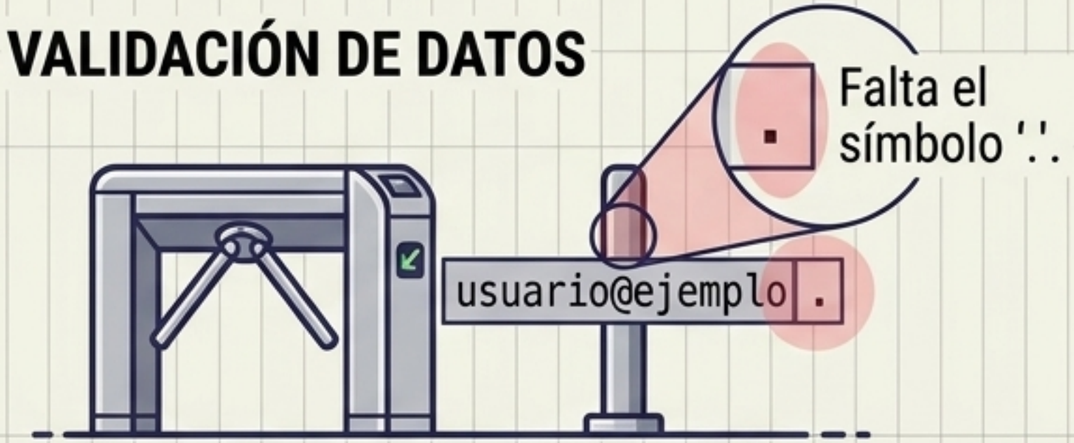
```
if tipo == 'suma':
    return sumar
```

En Python, una función se trata exactamente igual que un número o una cadena de texto.

Gemini Notebook

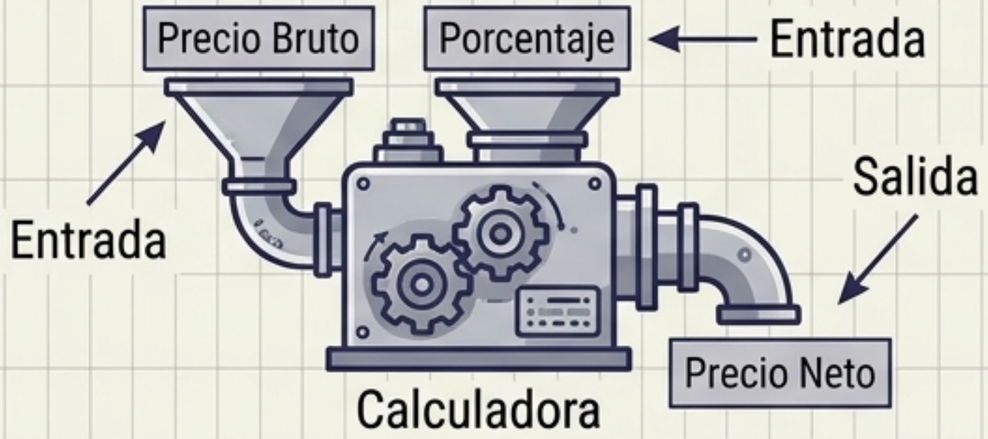
Casos de Uso Reales: Validación y Cálculos

VALIDACIÓN DE DATOS




```python
def validar_email(email):
    if "@" not in email:
        return False
    if "." not in email.split("@")[1]:
        return False
    return True
```

CÁLCULOS REUTILIZABLES



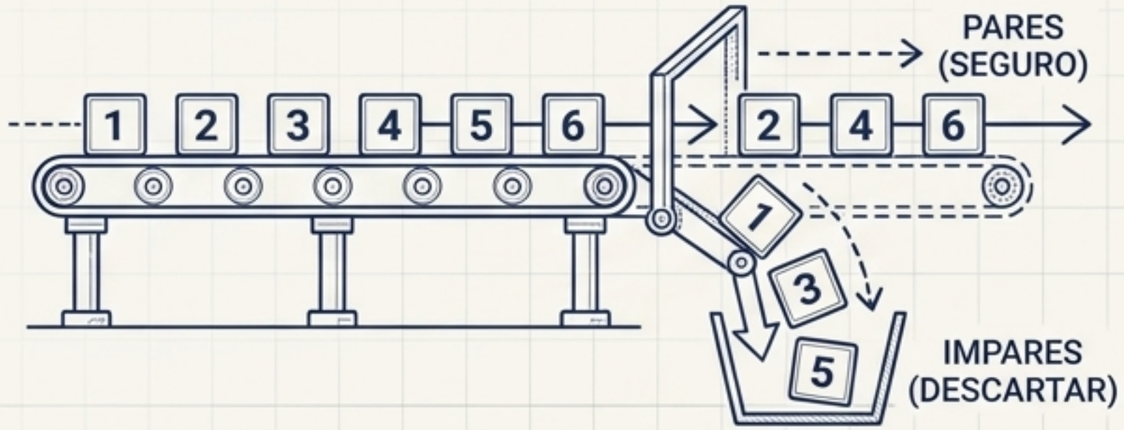

```python
def calcular_descuento(precio, porcentaje):
    descuento = precio * (porcentaje / 100)
    return precio - descuento

precio_camisa = calcular_descuento(50, 20)
```

Gemini Notebook

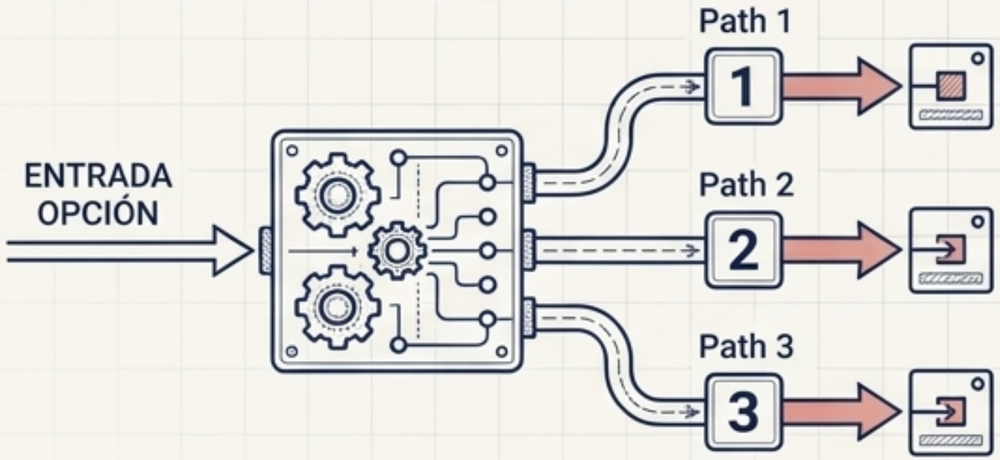
Casos de Uso Reales: Procesamiento y Menús

Procesamiento de Listas




```python
def filtrar_pares(numeros):
    pares = []
    for num in numeros:
        if num % 2 == 0:
            pares.append(num)
    return pares
```

Sistemas de Menú / UI




```python
def procesar_opcion(opcion):
    if opcion == "1":
        return "Ejecutando opción 1..."
    elif opcion == "2":
        return "Ejecutando opción 2..."
    elif opcion == "3":
        return "¡Hasta luego!"
```

Gemini Notebook

Diagnóstico de Errores: Expectativa vs. Realidad

Error Común (Expectativa)	Solución (Realidad)
Olvidar paréntesis. `print(saludar)` ← Devuelve el objeto en memoria `<function...>`	**Code:** `print(saludar())` ← Ejecuta y devuelve el resultado real.
Modificar listas por defecto. `def agregar(item, lista=[]):` Guarda el estado entre llamadas (comportamiento mutante).	**Code:** `def agregar(item, lista=None):` Usar `None` y crear la lista dentro de la función.
Falta de Return. `resultado = num * 2` Sin return, la función devuelve `None` silenciosamente.	**Code:** `return resultado` Asegura la entrega del dato.
Confundir Print con Return. `print(a + b)` Solo muestra en pantalla, no permite guardarlo en variable.	**Code:** `return a + b` Entrega el dato utilizable al programa.

Gemini Notebook

Check-list del Artesano: Buenas Prácticas

- [x] Nombres descriptivos y verbos en minúscula (`snake_case`). Funciones pequeñas que hacen una sola cosa bien.
- [x] Usa parámetros con nombre para llamadas complejas, mejorando la legibilidad.
- [x] Los parámetros por defecto siempre al final de la definición.
- [x] Nunca uses tipos mutables (listas/diccionarios) como valores por defecto en parámetros.
- [x] Evita interactuar con variables globales. Pasa datos como parámetros y usa `return`.
- [x] Si tu función no tiene un `return` explícito, devolverá `None` implícitamente.
- [x] Usa `Docstrings` (triple comillas) como estándar innegociable de documentación.

Gemini Notebook

El Camino Continúa



Estás aquí:
Dominio de Funciones

Próxima parada:
Funciones Lambda y
y Programación Funcional

Has dominado la arquitectura fundamental. El siguiente nivel de elegancia te espera en el ecosistema funcional.

📥 **Llévate el cheatsheet de Python (gratis).**

PDF de referencia rápida con tipos, bucles, funciones y errores comunes.

🚀 **¿Quieres aprender Python en orden y sin saltos?**

Domina decoradores, generadores y proyectos reales desde la terminal a producción en el curso y libro Python a fondo.

Gemini Notebook